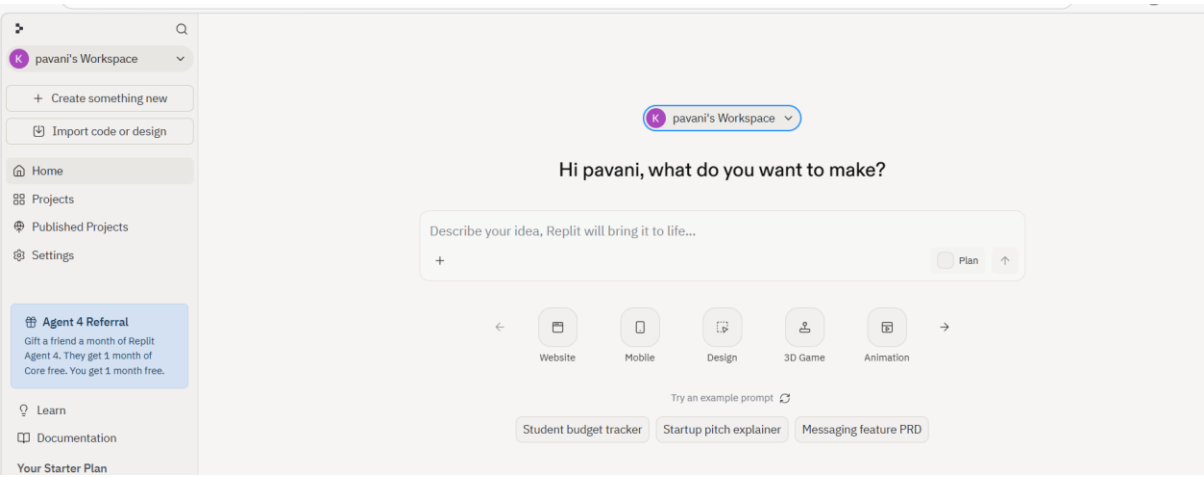
LAB-24.1(AIAC)

2303A51494

B27

Task 1 – Environment Setup

OUTPUT:



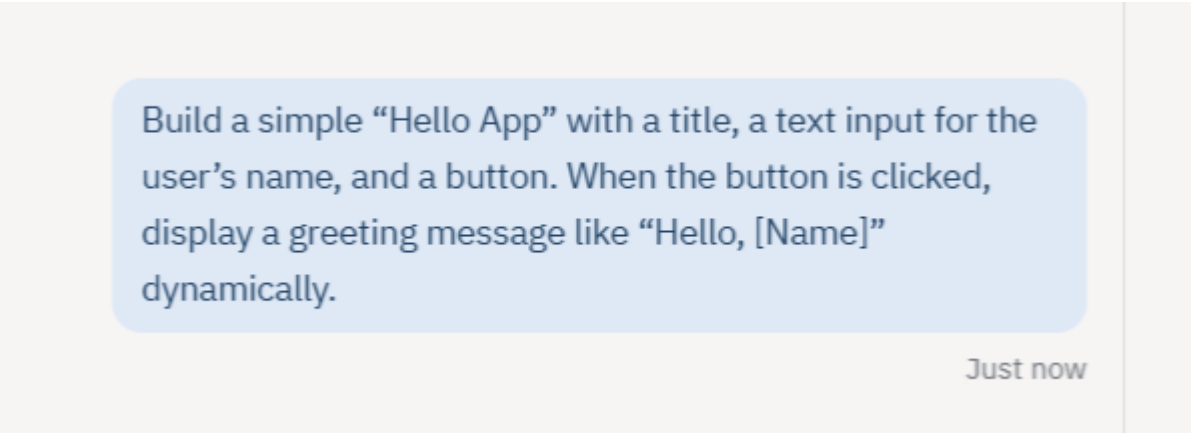
EXPLANATION:

- created a free account on Replit and successfully accessed the workspace dashboard.
- I explored the available project templates for mobile and web app development.
- The environment is now ready to create and run projects.

Task 2 – Basic Mobile App

PROMPT:

Build a simple “Hello App” with a title, a text input for the user’s name, and a button. When the button is clicked, display a greeting message like “Hello, [Name]” dynamically.



CODE:

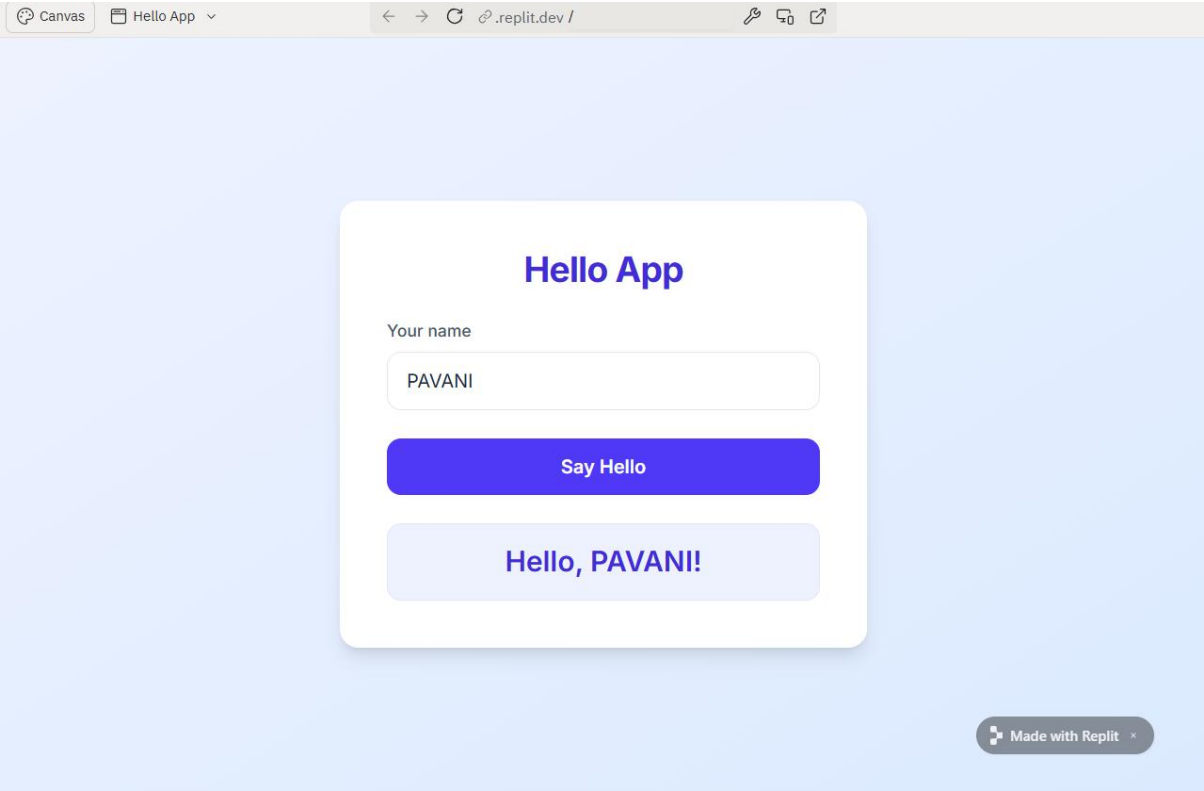
```
11        <div className="min-h-screen w-full flex items-center justify-center bg-gray-50">
12          <div className="text-center">
13            <h1 className="text-2xl font-bold text-gray-900">Replit Agent is building...</h1>
14            <p className="mt-2 text-sm text-gray-600">Your app will appear here once it's ready.</p>
20      <div className="min-h-screen flex items-center justify-center bg-gradient-to-br from-indigo-50 to-blue-100">
21        <div className="bg-white rounded-2xl shadow-lg p-10 w-full max-w-md flex flex-col gap-6">
22          <h1 className="text-3xl font-bold text-indigo-700 text-center tracking-tight">
23            Hello App
24          </h1>
25
26          <div className="flex flex-col gap-2">
27            <label htmlFor="name-input" className="text-sm font-medium text-gray-600">
28              Your name
29            </label>
30            <input
31              id="name-input"
32              type="text"
33              value={name}
34              onChange={(e) => setName(e.target.value)}
35              onKeyDown={handleKeyDown}
36              placeholder="Enter your name..."
37              className="w-full px-4 py-3 border border-gray-200 rounded-xl text-gray-800 placeholder-gray-400 focus:outline-none focus:ring-2 focus:ring-indigo-400 focus:border-transparent transition"
38            />
39          </div>
40
41          <button
42            onClick={handleGreet}
43            className="w-full py-3 bg-indigo-600 hover:bg-indigo-700 active:bg-indigo-800 text-white font-semibold rounded-xl transition-colors duration-150 cursor-pointer"
44          >
45            Say Hello
```

```
15 53        </div>
16 54      </div>
17 55    );
18 56  }
19 57
20       function Router() {
21         return (
22           <Switch>
23             <Route path="/" component={Home} />
24             <Route component={NotFound} />
25           </Switch>
26         );
27       }
28
29       function App() {
30         return (
31           <QueryClientProvider client={queryClient}>
32             <TooltipProvider>
33               <WouterRouter base={import.meta.env.BASE_URL.replace(/\/$/, "")}>
34                 <Router />
35               </WouterRouter>
36               <Toaster />
37             </TooltipProvider>
38           </QueryClientProvider>
39         );
40       }
41
42 58  export default App;
```

artifacts > hello-app > src > main.tsx

```
import { createRoot } from "react-dom/client";
import App from "./App";
import "./index.css";

createRoot(document.getElementById("root")!).render(<App />);

```

OUTPUT:



EXPLANATION:

- A simple Hello App is built with a title, an input field for the user's name, and a button to display a dynamic greeting message.
- AI assistance is used to generate the boilerplate code and explain the basic structure of the application.
- This helps in understanding how different components of the app work together.

Task 3 – Adding Features with AI Assistance

PROMPT:

Enhance the existing app by adding a navigation bar with Home, About, and Contact pages, along with a user registration form that handles input and submission. Apply basic styling using CSS to improve the UI. Also, generate clear inline comments for each function to explain its purpose and functionality.

CODE:

artifacts/hello-app/src/App.tsx

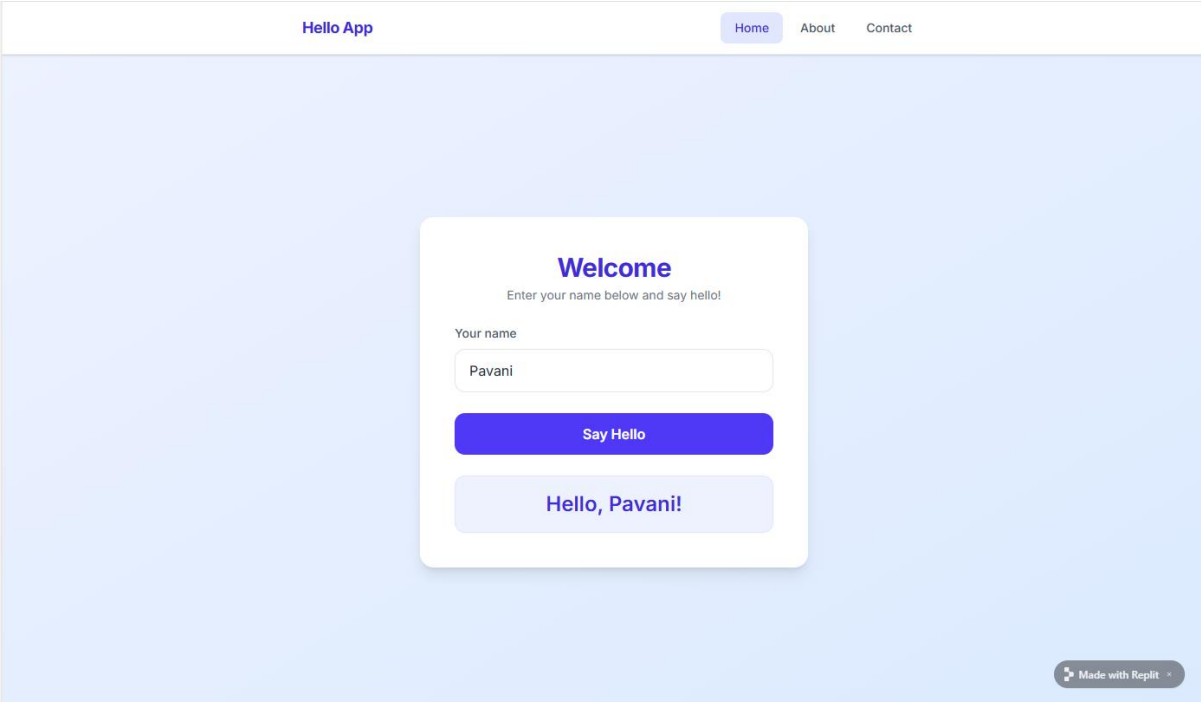
-51 +51

```
-  import { useState } from "react";
+  import { Switch, Route, Router as WouterRouter } from "wouter";
+  import NavBar from "@/components/NavBar";
+  import Home from "@/pages/Home";
+  import About from "@/pages/About";
+  import Contact from "@/pages/Contact";

-  function App() {
-    const [name, setName] = useState("");
-    const [greeting, setGreeting] = useState("");
-
-    function handleGreet() {
-      if (name.trim()) {
-        setGreeting(`Hello, ${name.trim()}!`);
-      }
-    }
-
-    function handleKeyDown(e: React.KeyboardEvent<HTMLInputElement>) {
-      if (e.key === "Enter") {
-        handleGreet();
-      }
-    }
-
+  /**
+   * NotFound — fallback page rendered when no route matches the current URL.
+   * Gives the user a clear message and a visual cue that the page doesn't exist.
+   */
+  function NotFound() {
    return (
```

```
11 function NotFound() {
12   return (
      <div className="min-h-screen flex items-center justify-center bg-gradient-to-br from-indigo-50 to-blue-100">
        <div className="bg-white rounded-2xl shadow-lg p-10 w-full max-w-md flex flex-col gap-6">
          <h1 className="text-3xl font-bold text-indigo-700 text-center tracking-tight">
            Hello App
          </h1>

          <div className="flex flex-col gap-2">
            <label htmlFor="name-input" className="text-sm font-medium text-gray-600">
              Your name
            </label>
            <input
              id="name-input"
              type="text"
              value={name}
              onChange={(e) => setName(e.target.value)}
              onKeyDown={handleKeyDown}
              placeholder="Enter your name..."
              className="w-full px-4 py-3 border border-gray-200 rounded-xl text-gray-800 placeholder-gray-400 focus:outline-none focus:ring-2 focus:ring-indigo-400 focus:border-transparent transition"
            />
          </div>

          <button
            onClick={handleGreet}
            className="w-full py-3 bg-indigo-600 hover:bg-indigo-700 active:bg-indigo-800 text-white font-semibold rounded-xl transition-colors duration-150 cursor-pointer"
          >
            Say Hello
          </button>
```

```
15       <p className="text-gray-500 mt-3">Page not found.</p>
16     </div>
17   );
18 }
19
20 /**
21  * Router — maps URL paths to their corresponding page components.
22  * Uses wouter's Switch so only the first matching Route is rendered.
23  */
24 function Router() {
25   return (
26     <Switch>
27       <Route path="/" component={Home} />
28       <Route path="/about" component={About} />
29       <Route path="/contact" component={Contact} />
30       <Route component={NotFound} />
31     </Switch>
32   );
33 }
34
35 /**
36  * App — root component of the application.
37  * Sets up the wouter router with the artifact's base path and composes
38  * the NavBar with the page content area into a full-screen layout.
39  */
40 function App() {
41   return (
42     // WouterRouter is given the BASE_URL so routing works under any deploy prefix
43     <WouterRouter base={import.meta.env.BASE_URL.replace(/\/$/, "")}>
44       {/* Outer container fills the viewport and stacks the nav above the page */}
45       <div className="min-h-screen flex flex-col bg-gradient-to-br from-indigo-50 to-blue-100">
46         {/* Persistent navigation bar rendered on every page */}
47         <NavBar />
48
49         {/* Main content area — each page component renders inside here */}
50         <main className="flex flex-col flex-1">
51           <Router />
52         </main>
53       </div>
54     </WouterRouter>
55   );
```

OUTPUT:



EXPLANATION:

- The application is enhanced by adding a navigation bar, a user registration form, and basic styling to improve the user interface.
- Form handling is implemented to capture and process user input effectively.
- AI assistance is used to add inline documentation, making the code easier to understand and maintain.

Task 4 – Testing on Mobile

PROMPT:

Run the application in Replit’s mobile preview mode and test its responsiveness across different screen sizes. Identify any UI alignment or layout issues and suggest improvements. Use AI assistance to fix these issues and ensure a clean, mobile-friendly design.

> Run the application in Replit’s mobile preview mode and test its responsiveness across different screen sizes. Identify any UI alignment or layout issues and suggest improvements. Use AI assistance to fix these issues and ensure a clean, mobile-friendly design.
>
> Just now

CODE:

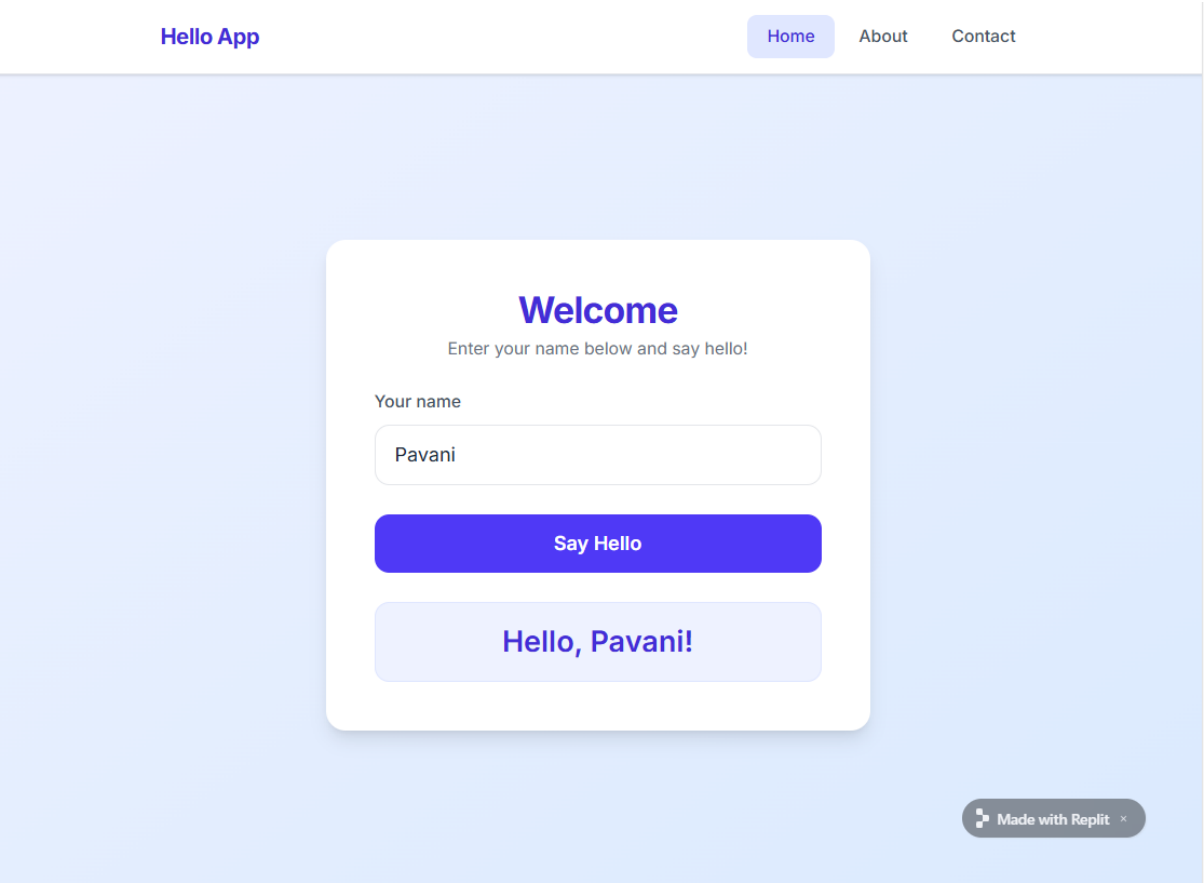
Preview | Diff: App.tsx | main.tsx | Diff: App.tsx | Diff: NavBar.tsx | Invite | Publish

```
 * NavBar component — renders the top navigation bar with links to all pages.
 * Uses wouter's `useLocation` hook to highlight the currently active route.
 * Fully responsive: stacks gracefully on narrow screens and uses touch-friendly tap areas.
 */
export default function NavBar() {
  // Get the current pathname so we can apply an active style to the matching link
  const [location] = useLocation();

  /**
   * Returns the CSS class string for a nav link.
   * Adds an "active" class when the link's path matches the current location.
   * navLinkClass — returns the Tailwind class string for a nav link.
   * Applies an active highlight when the link's path matches the current location.
   */
  function navLinkClass(path: string): string {
    const base =
      "px-4 py-2 rounded-lg text-sm font-medium transition-colors duration-150";
      "px-3 py-1.5 sm:px-4 sm:py-2 rounded-lg text-sm font-medium transition-colors duration-150 whitespace-nowrap";
    const active = "bg-indigo-100 text-indigo-700";
    const inactive = "text-gray-600 hover:bg-gray-100 hover:text-gray-900";
    return `${base} ${location === path ? active : inactive}`;
-5 +5

  return (
    <nav className="w-full bg-white border-b border-gray-200 shadow-sm">
      <div className="max-w-3xl mx-auto px-6 py-3 flex items-center justify-between">
        {/* Brand / logo area */}
        <span className="text-indigo-700 font-bold text-lg tracking-tight">
      <div className="max-w-3xl mx-auto px-4 sm:px-6 py-3 flex items-center justify-between gap-4">
        {/* Brand / logo — shrinks slightly on very small screens */}
        <span className="text-indigo-700 font-bold text-base sm:text-lg tracking-tight shrink-0">
          Hello App
        </span>

        {/* Navigation links */}
        <div className="flex items-center gap-1">
        {/* Navigation links — always in a row but compact on mobile */}
        <div className="flex items-center gap-0.5 sm:gap-1 overflow-x-auto">
          <Link href="/" className={navLinkClass("/")}>
            Home
          </Link>
```

artifacts/hello-app/src/pages/About.tsx

```diff
-3 +4
 1  1  /**
 2  2   * About page component — describes the purpose and features of the Hello App.
 3  3   * This is a purely presentational page with no interactive state.
    4 + * Responsive layout: padding scales up from mobile to desktop.
 4  5   */
 5  6  export default function About() {
 6  7    return (
 7       <div className="flex flex-col items-center justify-center flex-1 py-20 px-4">
 8         <div className="bg-white rounded-2xl shadow-lg p-10 w-full max-w-2xl flex flex-col gap-0">
    8 +  <div className="flex flex-col items-center justify-center flex-1 py-10 sm:py-20 px-4">
    9 +    <div className="bg-white rounded-2xl shadow-lg p-6 sm:p-10 w-full max-w-2xl flex flex-col gap-0">
 9 10         {/* Page heading */}
10 11         <div className="text-center">
11             <h1 className="text-3xl font-bold text-indigo-700 tracking-tight">
   12 +        <h1 className="text-2xl sm:text-3xl font-bold text-indigo-700 tracking-tight">
12 13             About
13 14           </h1>
14 15           <p className="text-gray-500 mt-1 text-sm">
-2 +2
16 17           </p>
17 18         </div>
18 19
19             {/* Feature list section */}
20             <div className="flex flex-col gap-4">
   20 +        {/* Feature list — stacks vertically on all screen sizes */}
   21 +        <div className="flex flex-col gap-3 sm:gap-4">
21 22           <FeatureCard
22 23             icon="👋"
23 24             title="Personalised Greetings"
-2 +3
40 41  }
41 42
42 43  /**
43       * FeatureCard — a small presentational card used to describe a single feature.
44       * Accepts an icon emoji, a title, and a description string.
   44 + * FeatureCard — a small presentational card describing a single app feature.
   45 + * Uses flex layout with a fixed-size icon and wrapping text so it looks good
   46 + * at any screen width without truncation.
45 47   */
46 48  function FeatureCard({
47 49    icon,
-5 +6
53 55    description: string;
54 56  }) {
55 57    return (
56       <div className="flex items-start gap-4 p-4 bg-indigo-50 border border-indigo-100 rounded-xl">
57         {/* Decorative icon */}
58         <span className="text-2xl">{icon}</span>
   58 +  <div className="flex items-start gap-3 sm:gap-4 p-4 bg-indigo-50 border border-indigo-100 rounded-xl">
   59 +    {/* Decorative icon — fixed width prevents layout shift */}
   60 +    <span className="text-xl sm:text-2xl shrink-0 mt-0.5">{icon}</span>
59 61
60             <div>
   62 +        <div className="min-w-0">
   63 +          {/* min-w-0 allows text to wrap properly inside the flex child */}
61 64           <h3 className="font-semibold text-indigo-800 text-sm">{title}</h3>
62             <p className="text-gray-600 text-sm mt-0.5">{description}</p>
   65 +        <p className="text-gray-600 text-sm mt-0.5 leading-relaxed">{description}</p>
63 66         </div>
64 67       </div>
65 68     );
```

OUTPUT:


Hello App
Home
About
Contact
Welcome
Enter your name below and say hello!
Your name
Pavani
Say Hello
Hello, Pavani!
Made with Replit

Issues identified:

Issue	Where	Fix applied
Card padding too wide (40px) on mobile, left almost no content space	All pages	Reduced to 24px on mobile, 40px on `sm+` screens
`py-20` top/bottom padding pushed content too far down on small screens	All pages	Reduced to `py-10` on mobile, `py-20` on `sm+`
2-column name grid was too cramped at 375px	Contact form	Now stacks to 1 column on mobile, 2 columns on `sm+`
NavBar links had no room to breathe between brand name	NavBar	Tighter mobile padding, added `gap` between elements
Inputs and buttons too short for reliable touch taps	All forms	Added `min-h-[44px]` — Apple's recommended touch target size
`font-size: 14px` on inputs caused iOS Safari to auto-zoom on focus	All inputs	Set to `text-base` (16px) to prevent the zoom
Missing `autocomplete` attributes	Contact form	Added `given-name`, `family-name`, `email`, `new-password`

The app now scales cleanly from 375px phones up through tablets and desktop.

Checkpoint made just now

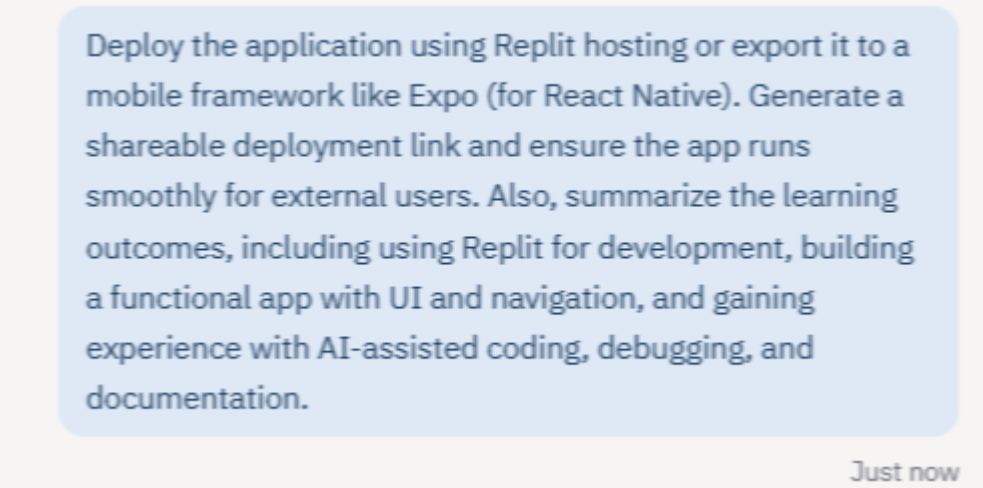
EXPLANATION:

- The application is tested in Replit's mobile preview mode to ensure it works smoothly on different screen sizes.
- UI alignment and layout issues are identified during testing.
- AI assistance is used to fix these issues and improve responsiveness for a better user experience.

Task 5 – Deployment

PROMPT:

Deploy the application using Replit hosting or export it to a mobile framework like Expo (for React Native). Generate a shareable deployment link and ensure the app runs smoothly for external users. Also, summarize the learning outcomes, including using Replit for development, building a functional app with UI and navigation, and gaining experience with AI-assisted coding, debugging, and documentation.



OUTPUT:

44px minimum touch targets

- Fixed a common iOS Safari issue where inputs below 16px font size trigger an unwanted auto-zoom

4. AI-Assisted Coding, Debugging, and Documentation

- Used AI assistance to **scaffold components quickly** and wire up routing without boilerplate guesswork
- Identified and resolved **layout bugs** by taking screenshots at multiple viewport sizes and applying targeted fixes
- Generated **clear inline comments** on every function explaining its purpose and behaviour — making the codebase self-documenting
- Iterated rapidly: build → preview → test → fix, all within a single session

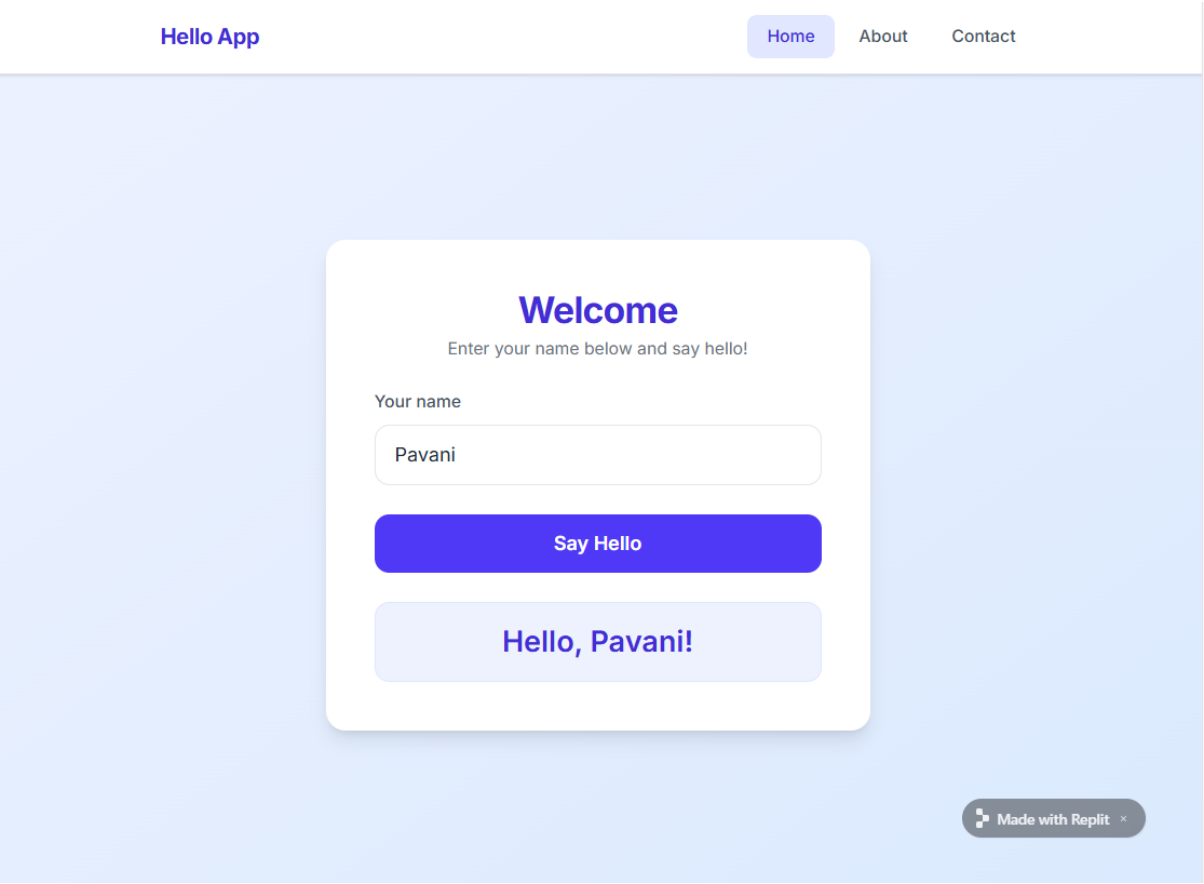
Worked for 19 seconds

Ready to share? Publish this App in one click.

Publish to domain ✓ Available

greeting-display--2303a51494 .replit.app

Configure myself | Publish now



EXPLANATION:

- The application is deployed using Replit hosting or exported to a mobile framework to make it accessible online.
- The deployment link is shared with others for testing and feedback.
- This task helps in understanding app deployment, building functional mobile applications, and gaining experience with AI-assisted development, debugging, and documentation.